

第4章 函数

上一章的游戏能玩，但所有代码挤在一起。想再玩一局就得把整段复制一遍，想改提示语得在好几处改。这一章把它拆成几个函数，顺便学会怎么把代码组织成能反复使用的零件。

4.1 先把游戏拆开

本节任务：把猜数字游戏拆成几个各管一件事的函数，让它能连玩好几局并统计成绩。

```
1 import random
2
3 def make_answer(low=1, high=100):
4     """随机想一个 low 到 high 之间的数"""
5     return random.randint(low, high)
6
7 def judge(guess, answer):
8     """判断猜得如何，返回提示语和是否猜中"""
9     if guess == answer:
10         return "猜中了", True
11     elif guess < answer:
12         return "小了", False
13     else:
14         return "大了", False
15
16 def play_once(low=1, high=100):
17     """玩一局，返回用了多少次"""
18     answer = make_answer(low, high)
19     count = 0
20     while True:
21         guess = int(input(f"猜一个 {low} 到 {high} 的数: "))
22         count = count + 1
23         msg, ok = judge(guess, answer) # 一次收下两个返回值
24         print(msg)
25         if ok:
26             print(f"用了 {count} 次")
27             return count
28
```

```
29 records = []
30 for round_no in range(1, 4): # 连玩三局
31     print(f"--- 第 {round_no} 局 ---")
32     records.append(play_once())
33
34 print(f"三局分别用了 {records} 次，平均 {sum(records)/len(records):.1f} 次")
```

玩完三局的输出大致是：

三局分别用了 [4, 6, 7] 次，平均 5.7 次

和上一章比，代码行数没少多少，好处却很明显。判断逻辑只写在 `judge` 里一处，改提示语只改那里。想改成猜 1 到 1000，调用时传参数即可，不用翻遍代码。玩三局只要循环调用，不用复制粘贴。下面把函数的各个部分讲清楚。

4.2 定义和调用

本节任务：会自己定义函数并调用它。

定义函数用 `def`，后面跟函数名、圆括号、冒号，下一行缩进写函数体：

★核心

```
1 def say_hello():
2     print("你好")
3
4 say_hello() # 调用，输出 你好
```

定义只是把这段代码起个名字存起来，不会执行。写上函数名加圆括号才是**调用**，这时里面的代码才真正运行。圆括号不能省，只写 `say_hello` 不会执行任何东西。

函数名的规矩和变量名一样，建议用动词开头，让人一看就知道它做什么，比如 `make_answer` 比 `answer` 好。

4.3 参数，把数据传进去

本节任务：搞清位置参数、默认参数和关键字参数三种传法。

圆括号里可以写**参数**，调用时把数据传进去：

```
1 def greet(name):
2     print(f"你好，{name}")
3
4 greet("小明") # 你好，小明
5 greet("小红") # 你好，小红
```

同一个函数配上不同的数据就能干不同的活，这是函数最大的价值。

多个参数按顺序对应，这叫**位置参数**：

```
1 def power(base, exp):
2     return base ** exp
3
4 print(power(2, 10)) # 1024 base 是 2, exp 是 10
5 print(power(10, 2)) # 100 顺序换了，含义就变了
```

参数可以给**默认值**，调用时不传就用默认的：

★核心

```
1 def make_answer(low=1, high=100):
2     return random.randint(low, high)
3
4 make_answer() # 用默认的 1 到 100
5 make_answer(1, 10) # 改成 1 到 10
```

参数多了以后按顺序传容易记混，可以写上名字，这叫**关键字参数**，此时顺序无所谓：

```
1 make_answer(high=10, low=1) # 写了名字，顺序随便
```

有个规矩要记住，带默认值的参数必须放在没默认值的后面。写成 `def f(a=1, b)` 会直接报语法错误。

4.4 返回值，把结果拿出来

本节任务：分清打印和返回，会返回多个值。

函数算完的结果要用 **return** 送出来，调用方才能接住。这是新手容易含糊的地方，★核心**打印和返回完全是两回事**。打印只是显示给人看，返回才是把值交给程序继续用：

```
1 def add_print(a, b):
2     print(a + b) # 只显示，不交出去
3
4 def add_return(a, b):
5     return a + b # 交出去
6
7 x = add_print(1, 2) # 屏幕出现 3，但 x 是 None
8 y = add_return(1, 2) # 屏幕什么也没有，但 y 是 3
9 print(x, y) # None 3
```

没写 `return` 的函数，返回的是 `None`，表示没有值。

执行到 `return` 函数就立刻结束，后面的代码不再运行。所以 `return` 也常用来提前退出：

```
1 def check(score):
2     if score < 0:
3         return "分数不能是负的" # 直接结束，不往下走
4     return "分数正常"
```

一次返回多个值，用逗号隔开就行，接的时候也用逗号：

★核心

```
1 def judge(guess, answer):
2     if guess == answer:
3         return "猜中了", True
4     return "没中", False
5
6 msg, ok = judge(50, 68) # 一次接两个
7 print(msg, ok) # 没中 False
```

开头游戏里的 `msg, ok = judge(...)` 就是这么用的。

4.5 函数里的变量出不来

本节任务：理解作用域，避开一类难查的错。

函数里创建的变量只在函数内部有效，出了函数就不存在，这叫**局部变量**：

★核心

```
1 def f():
2     inside = 10
3     print(inside) # 10 在函数内能用
4
5 f()
6 # print(inside) # 报 NameError, 外面不认识它
```

反过来，函数内部可以读到外面的变量，但直接赋值会创建一个同名的新局部变量，不会影响外面的：

```
1 count = 0
2
3 def add():
4     count = count + 1 # 这里会报错
5
```

```
6 # add() # UnboundLocalError
```

这个报错看着奇怪，原因是函数里一旦对 `count` 赋值，Python 就认定 `count` 是局部变量，而等号右边又要先读它，此时它还没有值。正确做法不是去改外面的变量，而是把值传进去、把结果返回来：

```
1 def add(count):
2     return count + 1
3
4 count = 0
5 count = add(count) # 1
```

这样函数只依赖参数、只通过返回值影响外界，谁调用它都能预期结果。养成这个习惯，程序会好懂也好查错。

4.6 给函数写说明

本节任务：用一句话说明函数做什么。

函数体第一行可以写一个字符串，用来说明这个函数干什么，叫**文档字符串**。开头游戏里每个函数下面那行引号就是它：

```
1 def make_answer(low=1, high=100):
2     """随机想一个 low 到 high 之间的数"""
3     return random.randint(low, high)
4
5 print(make_answer.__doc__) # 随机想一个 low 到 high 之间的数
6 help(make_answer) # 也能看到它
```

写不写不影响运行，但函数一多，说明就是自己和别人的救命稻草。一句话讲清做什么即可。

4.7 什么时候该写成函数

本节任务：知道拆函数的时机和标准。

有两个明显信号。一是同样的代码复制了第二遍，二是一段代码长到得往上翻才能看懂。这两种情况都该拆。

拆的标准是**一个函数只做一件事**，名字能概括这件事。判断大小就只判断，不要顺带打印和计数。这样每个函数都短、好测、好改。开头的游戏就是按这个拆的，`make_answer` 只负责出题，`judge` 只负责判断，`play_once` 负责把一局串起来。

拆好之后还有个好处，想改玩法很容易。比如想让电脑来猜你心里的数，只要重写 `play_once` 里问答那部分，`judge` 完全不用动。

4.8 小结

这一章把散落的代码组织成了零件。用 `def` 定义函数，写上名字加圆括号才是调用。参数让同一个函数处理不同数据，可以按位置传、给默认值、也可以写名字传。`return` 把结果交给调用方，和打印是两回事，没写 `return` 就返回 `None`，用逗号可以一次返回多个值。函数里的变量出了函数就不存在，要影响外界应该靠参数和返回值而不是去改外部变量。函数体第一行的字符串是文档说明。代码重复了或者太长了，就该拆成函数，一个函数只做一件事。

到这里，变量、容器、分支循环、函数这几样最基本的东西都齐了，已经足够写出像样的小程序。

本章知识点

- **def** —— 定义函数的关键字，定义本身不执行，调用时才运行
- **调用** —— 函数名加圆括号，圆括号不能省
- **位置参数** —— 按顺序对应的参数，顺序错了含义就变
- **默认参数** —— 给参数预设值，调用时不传就用它，必须放在无默认值参数的后面
- **关键字参数** —— 调用时写上参数名，此时顺序无所谓
- **return** —— 把结果交给调用方，执行到它函数立刻结束
- **打印与返回** —— 打印只给人看，返回才把值交给程序继续用
- **None** —— 没有值，函数没写 `return` 时返回的就是它
- **多返回值** —— 用逗号隔开一次返回多个，接收时也用逗号
- **局部变量** —— 函数里创建的变量，出了函数就不存在
- **文档字符串** —— 函数体第一行的字符串，说明这个函数做什么
- **一个函数只做一件事** —— 拆分函数的标准，名字要能概括这件事